



PIT ERPNextSync

Externe ERP-Systeme mit ERPNext verbinden — Eine bidirektionale Datensynchronisationsbrücke

1. Was ist das?

PIT ERPNextSync ist eine **Frappe/ERPNext Custom App** der PIT IT GmbH, die Stammdaten und transaktionale Datensätze von **externen ERP-Systemen** nach **ERPNext** synchronisiert. Sie fungiert als konfigurierbare, automatisierte Brücke zwischen zwei ERP-Welten.

Die App verbindet sich direkt mit der Quelldatenbank — entweder

- **Microsoft SQL Server** (via `pymssql`), oder
- **4D** (via `p4d`)

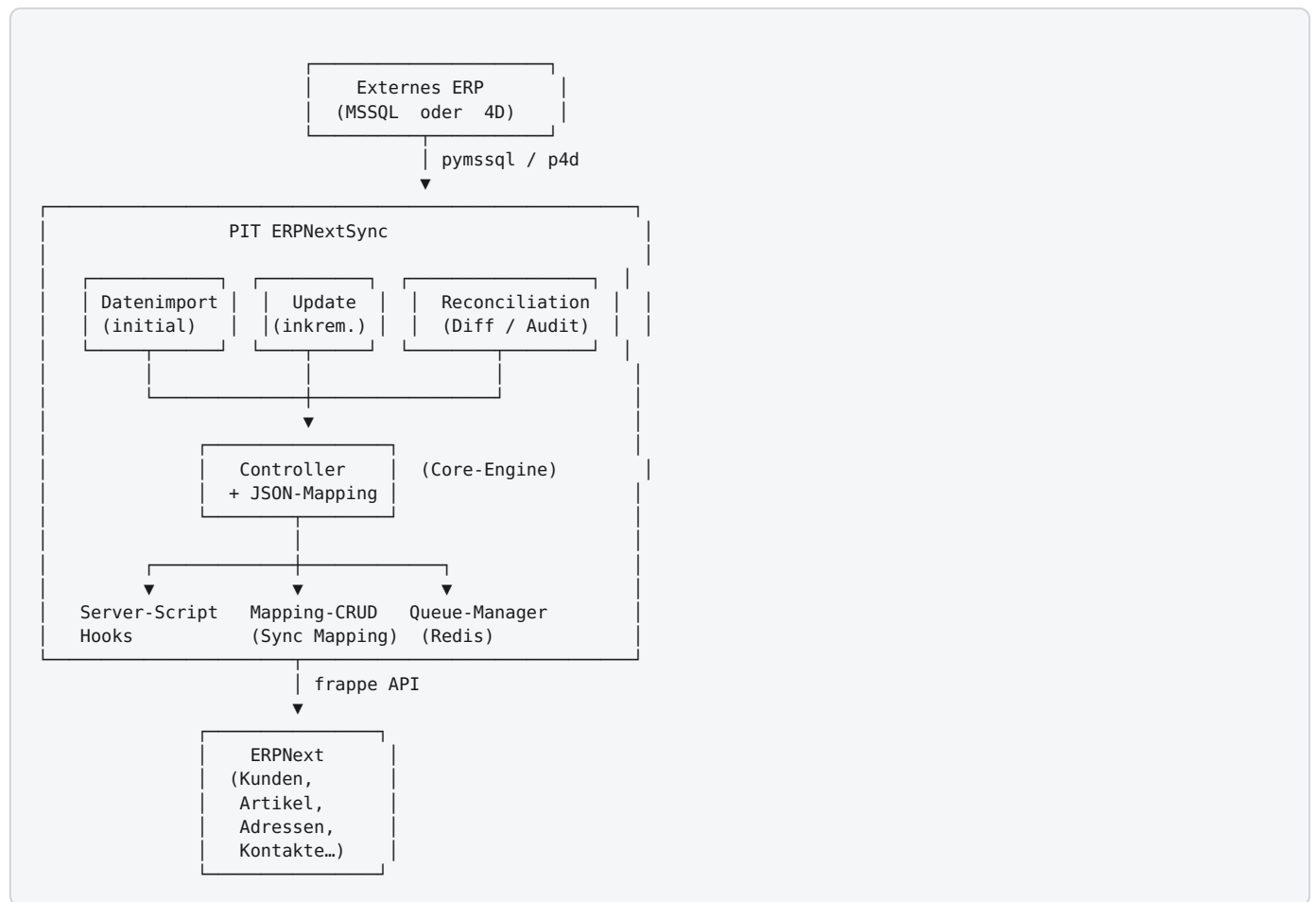
— liest Quelldatensätze, transformiert sie über ein **deklaratives JSON-Mapping** und erstellt oder aktualisiert ERPNext-Dokumente. Keine Middleware, keine externen ETL-Tools.

2. Welches Problem löst die App?

Viele Unternehmen betreiben ein externes ERP-System operativ und nutzen ERPNext parallel für E-Commerce, CRM oder Finanzkonsolidierung. Manuelles Übertragen von Daten zwischen beiden Systemen ist fehleranfällig und zeitaufwendig. PIT ERPNextSync automatisiert dies durch:

- **Einmaliger Bulk-Import** — Migration tausender Kunden, Artikel, Adressen und Kontakte in einem Durchlauf.
- **Inkrementelle, timestamp-basierte Updates** — nach dem Initialimport werden nur im Quellsystem geänderte Datensätze in ERPNext aktualisiert. Beide Systeme bleiben bei minimaler Last synchron.
- **Reconciliation (Abgleich)** — vergleicht die aktuelle JSON-Mapping-Definition mit den tatsächlich importierten Daten und meldet Abweichungen.

3. Architektur im Überblick



4. Kernfunktionen

4.1 Deklaratives JSON-Mapping

Die gesamte Transformation wird durch eine **einzige JSON-Struktur** gesteuert, die im DocType Sync Instance hinterlegt ist. Für das Mapping neuer Tabellen ist kein Python-Code erforderlich. Wichtigste Mapping-Features:

Feature	Beschreibung
Direktes Spalten-Mapping	sl_column → ERPNext-Feldname zuweisen
Fallback-Spalten	alt_key als Alternative, falls die primäre Spalte leer ist
Statische Standardwerte	Beliebiges Feld mit einem default-Literalwert belegen
Dynamische Feldvariablen	field_var zur Laufzeit auflösen (z. B. default_territory)
Querverweis-Mappings	Zuvor importierten Datensatz via mapped_value nachschlagen
Child-Tables	ERPNext-Untertabellen aus derselben Zeile oder via multiple_query befüllen
Post-Field-Variablen	Generierte Dokument-IDs erfassen, um spätere Dokumente zu verknüpfen
Multi-Dokument-Zeilen	Eine Quellzeile kann mehrere DocTypes erzeugen (z. B. Kunde + Adresse)
Pflichtfeld-Prüfung	Drei Stufen: optional, Dokument überspringen wenn leer, Import abbrechen wenn leer

4.2 Import, Update, Reconciliation

Operation	Modul	Zweck
Datenimport	data_import.py (1.227 Zeilen)	Massen-Erstellung von ERPNext-Dokumenten aus dem Quellsystem. Unterstützt top-Limit, Typfilterung und automatische Telefonnummer-Formatierung.
Bulk-Update	update.py (734 Zeilen)	Timestamp-basierte inkrementelle Synchronisation. Vergleicht den Zeitstempel jedes gemappten Datensatzes mit dem Quellsystem; aktualisiert nur geänderte Dokumente. Läuft als gestaffelte Hintergrundjobs.
Reconciliation	reconcile.py (2.378 Zeilen)	Vergleicht die aktuelle JSON-Mapping-Definition mit den gespeicherten Sync Mapping-Einträgen. Erkennt neue, entfernte und geänderte Feld-Mappings ohne Zugriff auf das Quellsystem.

4.3 Zeitgesteuerte Automatisierung

Konfiguriert über die `hooks.py` Scheduler-Events kann jede Sync Instance in folgenden Intervallen ausgeführt werden: **alle** / **täglich** / **stündlich** / **wöchentlich** / **monatlich**. Der Scheduler-Zyklus führt aus:

1. `start_import()` — neue Datensätze importieren
2. `run_bulk_update()` — geänderte Datensätze aktualisieren

Langlaufende Jobs nutzen `frappe.enqueue(queue="long", timeout=600)` mit Redis-gestütztem Job-Tracking. Die Batch-Größe ist pro Instanz konfigurierbar.

4.4 Server-Script-Hooks

Die Sync Instance Hooks-Untertabelle erlaubt das Anhängen von **Server-Scripts**, die *vor* und *nach* Import- und Update-Läufen ausgeführt werden. Dies ermöglicht benutzerdefinierte Validierungen, Datenanreicherungen oder Integrationen mit anderen Systemen, ohne die Sync-Kernlogik ändern zu müssen.

5. Wichtige DocTypes

DocType	Rolle
Sync Instance	Zentrale Konfiguration: DB-Zugangsdaten, Treiberauswahl, JSON-Mapping-Definition, Scheduler-Intervall, Aktivierungsschalter, Batch-Größe und Feldvariablen-Speicher.
Selectline Table Mapping	Untertabelle von Sync Instance. Jede Zeile enthält ein Mapping auf Tabellenebene (das JSON, das definiert, wie eine Quelltable zu einem ERPNext-DocType wird).
Sync Mapping	Verfolgt die Beziehung Quelldatensatz → ERPNext-Dokument . Identifiziert durch die Mapping-ID: <code><Instanz>:<Tabelle>:<Primärschlüssel></code>
Sync Mapping Entry	Untertabelle von Sync Mapping. Eine Zeile pro tatsächlich nach ERPNext geschriebenem Feld. Wird von der Reconciliation verwendet.
Sync Instance Hooks	Untertabelle von Sync Instance. Verweist auf Server-Script-Dokumente für Before/After-Import/Update-Hooks.
PIT ERPNextSync Settings	Singleton-DocType für globale Einstellungen (z. B. Cascade-Delete-Schalter).

6. Vorteile & Alleinstellungsmerkmale

- **No-Code-Mapping.** Neue Tabellen werden durch Bearbeitung eines JSON-Arrays im Sync-Instance-Formular hinzugefügt. Für routinemäßige Mapping-Arbeiten ist kein Python-Code erforderlich.
- **Dualer Datenbanktreiber.** Arbeitet nativ mit MSSQL und 4D über einen einheitlichen Dispatcher in `db_connect()`. SQL-Generierung, Spalten-Quoting und Fetch-Logik passen sich automatisch pro Treiber an.
- **Idempotente Synchronisation.** Das eindeutige Mapping-ID-Format (Instanz:Tabelle:Primärschlüssel) stellt sicher, dass derselbe Quelldatensatz nie doppelt in ERPNext angelegt wird. Updates werden auf das bestehende Dokument angewendet.
- **Inkrementelle, timestamp-basierte Updates.** Nach dem Initialimport werden nur Datensätze aktualisiert, deren `last_changed`-Zeitstempel neuer als die letzte Synchronisation ist. Minimiert die Datenbanklast auf beiden Seiten.
- **Batch-Verarbeitung mit Queue-Isolation.** Große Imports und Updates werden in konfigurierbare Batches aufgeteilt und als separate langlebige Redis-Jobs eingereiht. Der Job-Fortschritt wird verfolgt und ist einsehbar.
- **Integrierte Reconciliation.** Das System kann die aktuelle Mapping-Definition mit den gespeicherten Mapping-Einträgen abgleichen und Abweichungen aufdecken — ohne aktive Datenbankverbindung zum Quellsystem.
- **Server-Script-Hooks.** Beliebige Frappe-Server-Scripts können vor oder nach Import/Update ausgeführt werden. Ermöglicht benutzerdefinierte Geschäftslogik, Benachrichtigungen oder nachgelagerte Integrationen.
- **Telefonnummer-Normalisierung.** Formatiert Rufnummern automatisch ins internationale Format (+43... für Österreich, konfigurierbar nach Land).
- **Cascade-Delete-Unterstützung.** Wird ein Quelldatensatz im Quellsystem entfernt, kann das entsprechende ERPNext-Dokument optional gelöscht werden (konfigurierbar über globale Einstellungen).
- **Debug-freundlich.** Debug-Modus pro Instanz für ausführliches Logging, „Test Connection“-Button im Formular und alle Fehler im ERPNext Error Log (filterbar nach App-Name).
- **Konsistente Codebasis.** Ruff-formatiert (Tabs, doppelte Anführungszeichen), 110 Zeichen Zeilenlänge, Pre-Commit-Hooks und ein klarer Einstiegspunkt pro Operation.

7. Typischer Datenfluss

1. **Konfigurieren** — Sync Instance mit DB-Zugangsdaten, Treiber, Mapping-JSON, Wiederholungsintervall und zu importierenden Typen anlegen.
2. **Testen** — Auf *Test Connection* klicken, um die Datenbankverbindung zu prüfen.
3. **Importieren** — Manuellen Import mit kleinem top-Limit durchführen, um das Mapping zu validieren.
4. **Abgleichen** — Importergebnis mit der Mapping-Definition vergleichen (Reconciliation).
5. **Einplanen** — Scheduler aktivieren. Das System führt nun `start_import()` + `run_bulk_update()` im konfigurierten Intervall aus.
6. **Überwachen** — Error Log, Debug-Logs und Job-Queue auf Probleme prüfen.

8. Technologie-Stack

Schicht	Technologie
Laufzeit	Python 3.10+
Framework	Frappe (ERPNext Custom App)
MSSQL-Treiber	pymssql $\geq$ 2.2.0
4D-Treiber	p4d $\geq$ 1.8
Job-Queue	Redis (via <code>frappe.enqueue</code> + RQ)
Build-System	flit_core
Linting & Formatierung	Ruff (via Pre-Commit)
Parent-App-Abhängigkeit	pit_erpnext (stellt gemeinsamen Logger bereit)

9. Zusammenfassung

PIT ERPNextSync ist eine **praxiserprobte** Datensynchronisationsschicht zwischen externen ERP-Systemen und ERPNext. Die JSON-gesteuerte Mapping-Engine eliminiert benutzerdefinierten Code für routine Integrationsaufgaben, während die Dual-Treiber-Architektur, Batch-Verarbeitung und integrierte Reconciliation die App auch für Produktivumgebungen mit großen Datenmengen und gemischten Datenbank-Backends geeignet macht.

Dokumentation: `app_data/documentation/MAPPING_GUIDE.md` | Architektur: `AGENTS.md` | Quellcode: `pit_erpnextsync/scripts/`